tarefa_iteracoes_recursoes

September 7, 2026

1. fatorial

[11]:
```python
# a) algoritmo iterativo
def fatorial_iterativo(n):
    res = 1
    for i in range(2, n+1):
        res *= i
    return res

n = 5

print(f'fatorial iterativo de {n}: {fatorial_iterativo(n)}')

# b) algoritmo recursivo
def fatorial_recursivo(n):
    if n == 0:
        return 1
    return n * fatorial_recursivo(n-1)

n = 6

print(f'fatorial recursivo de {n}: {fatorial_recursivo(n)}')
```

```
fatorial iterativo de 5: 120
fatorial recursivo de 6: 720
```

2. soma dos elementos de uma lista

[15]:
```python
lista = [2, 5, 12, 10, 21]

# a) funcao iterativa que calcula a soma dos elementos
def soma_iterativa(lista):
    soma = 0
    for inteiro in lista:
        soma += inteiro
    return soma
print(f'soma iterativa dos elementos da lista: {soma_iterativa(lista)}')

# b) funcao recursiva
```

```
def soma_recursiva(lista):
    if not lista:
        return 0
    return lista[0] + soma_recursiva(lista[1:])
print(f'soma recursiva dos elementos da lista: {soma_recursiva(lista)}')
```

```
soma iterativa dos elementos da lista: 50
soma recursiva dos elementos da lista: 50
```

3. potência inteira

[24]:
```
# a) algoritmo iterativo
def potencia_int_iterativa(a, n):
    pot = a
    if n == 0:
        return 1
    for _ in range(2, n+1):
        pot *= a
    return pot

a, n = 3, 5
print(f'potência iterativa {a}**{n} = {potencia_int_iterativa(a, n)}')

# b) algoritmo recursivo
def potencia_int_recursiva(a, n):
    if n == 0:
        return 1
    return a * potencia_int_recursiva(a, n-1)

a, n = 3, 4
print(f'potência recursiva {a}**{n} = {potencia_int_recursiva(a, n)}')
```

```
potência iterativa 3**5 = 243
potência recursiva 3**4 = 81
```

4. sequência de fibonacci

[28]:
```
# a) solução iterativa
def fib_iterativo(n):
    if n <= 1:
        return n
    anterior, atual = 0, 1
    for _ in range(2, n+1):
        anterior, atual = atual, anterior + atual
    return atual

n = 11
print(f'fibonacci iterativo F({n}) = {fib_iterativo(n)}')
```

```
# b) solução recursiva
def fib_recursivo(n):
    if n <= 1:
        return n
    return fib_recursivo(n-1) + fib_recursivo(n-2)

n = 10
print(f'fibonacci recursivo F({n}) = {fib_recursivo(n)}')
```

```
fibonacci iterativo F(11) = 89
fibonacci recursivo F(10) = 55
```

observamos que a implementação recursiva é extremamente ineficiente, pois calcula a mesma informação diversas vezes. por exemplo, ao chamar `fib_recursivo(4)`, o programa executa `fib_recursivo(3) + fib_recursivo(2)`. o primeiro termo da soma então executa `fib_recursivo(2) + fib_recursivo(1)` e o segundo termo, `fib_recursivo(1) + fib_recursivo(0)`. uma última recursão ainda faz `fib_recursivo(2) = fib_recursivo(1) + fib_recursivo(0)` para daí começar a desempilhar as chamadas. nota que nesse exemplo simples, a mesma informação `fib_recursivo(2)` teve de ser computada duas vezes. o esquema abaixo ilustra os cálculos:

- fib_recursivo(4)
  - fib_recursivo(3)
    - fib_recursivo(2)
      - fib_recursivo(1)
      - fib_recursivo(0)
    - fib_recursivo(1)
  - fib_recursivo(2)
    - fib_recursivo(1)
    - fib_recursivo(0)

5. máximo de uma lista

[37]:
```
lista = [9.1, 2.4, 12.44, 4.0, 3.50, 5, 6.7]

# solucao iterativa
def max_iterativo(lista):
    maximo = lista[0]
    for i in range(1, len(lista)):
        if lista[i] > maximo:
            maximo = lista[i]
    return maximo

print(f'máximo iterativo da lista: {max_iterativo(lista)}')

# solucao recursiva
def max_recursivo(lista):
    if len(lista) == 1:
        return lista[0]
```

```
    else:
        maximo = max_recursivo(lista[1:])
        return maximo if maximo > lista[0] else lista[0]

print(f'máximo recursivo da lista: {max_recursivo(lista)}')
```

```
máximo iterativo da lista: 12.44
máximo recursivo da lista: 12.44
```

6. inverter uma string

[ ]:
```
# solucao iterativa
def inverte_iterativa(texto):
    invertida = ''
    for char in texto:
        invertida = char + invertida
    return invertida

texto = 'estou resfriado'
print(f'string invertida iterativa: {inverte_iterativa(texto)}')

# solucao recursiva
def inverte_recursiva(texto):
    n = len(texto)
    if n <= 1:
        return texto
    return inverte_recursiva(texto[1:]) + texto[0]

print(f'string invertida recursiva: {inverte_recursiva(texto)}')
```

```
string invertida iterativa: odairfser uotse
string invertida recursiva: odairfser uotse
```

7. máximo divisor comum

[47]:
```
# iterativo
def mdc_iterativo(a,b):
    while b != 0:
        a, b = b, a % b
    return a

a, b = 1071, 462
print(f'mdc iterativo ({a}, {b}) = {mdc_iterativo(a,b)}')

# recursivo
def mdc_recursivo(a,b):
    if b == 0:
        return a
    return mdc_recursivo(b, a % b)
```

```
print(f'mdc recursivo ({a}, {b}) = {mdc_recursivo(a,b)}')
```

```
mdc iterativo (1071, 462) = 21
mdc recursivo (1071, 462) = 21
```

8. soma dos algarismos de um número

[51]:
```
# iterativa
def soma_alg_iterativa(n):
    soma = 0
    while n > 0:
        soma += n % 10
        n //= 10
    return soma

n = 4729
print(f'soma iterativa dos algarismos de {n}: {soma_alg_iterativa(n)}')

# recursiva
def soma_alg_recursiva(n):
    if n < 10:
        soma = n
    return (n % 10) + soma_alg_iterativa(n // 10)

print(f'soma recursiva dos algarismos de {n}: {soma_alg_recursiva(n)}')
```

```
soma iterativa dos algarismos de 4729: 22
soma recursiva dos algarismos de 4729: 22
```

9. contagem de ocorrências

[57]:
```
# iterativa
def conta_iterativa(lista, x):
    contador = 0
    for numero in lista:
        if numero == x:
            contador += 1
    return contador

lista = [1, 1, 1, 2, 3, 4, 5, 6, 3]
x = 3
print(f'contagem iterativa de {x} em {lista}: {conta_iterativa(lista, x)}')

# recursiva
def conta_recursiva(lista, x):
    if not lista:
        return 0
    contador = 1 if lista[0] == x else 0
```

```
    return contador + conta_recursiva(lista[1:], x)

print(f'contagem recursiva de {x} em {lista}: {conta_recursiva(lista, x)}')
```

```
contagem iterativa de 3 em [1, 1, 1, 2, 3, 4, 5, 6, 3]: 2
contagem recursiva de 3 em [1, 1, 1, 2, 3, 4, 5, 6, 3]: 2
```

10. verificar se uma lista está ordenada

[71]:
```
# verifica se uma lista esta ordenada de forma crescente
# iterativa
def ordenada_iterativa(lista):
    n = len(lista)
    for i in range(0, n-1):
        if lista[i] > lista[i+1]:
            return f'a lista não está ordenada de forma crescente.'
            break
    return f'a lista está ordenada de forma crescente.'

lista = [1, 2, 3, 5, 6, 8, 4]
print(ordenada_iterativa(lista))

# recursiva
def ordenada_recursiva(lista):
    n = len(lista)
    if n <= 1:
        return f'a lista está ordenada de forma crescente.'
    elif lista[0] > lista[1]:
        return f'a lista não está ordenada de forma crescente.'
    return ordenada_recursiva(lista[1:])

print(ordenada_recursiva(lista))
```

```
a lista não está ordenada de forma crescente.
a lista não está ordenada de forma crescente.
```

11. potência rápida

[75]:
```
# recursao
def potencia_rapida(a, n):
    if n == 0:
        return 1
    elif n % 2 == 0:
        return potencia_rapida(a, n/2) * potencia_rapida(a, n/2)
    else:
        return a * potencia_rapida(a, (n-1)/2) * potencia_rapida(a, (n-1)/2)

a, n = 4, 5
print(f'potência rápida {a}**{n}: {potencia_rapida(a,n)}')
```

```
potência rápida 4**5: 1024
```

12. palíndromo

[88]:
```python
# recursao
def palindromo_recursivo(texto):
    n = len(texto)
    if n <= 1:
        return "a string é um palíndromo."
    else:
        if texto[0] != texto[-1]:
            return "a string não é um palíndromo."
        return palindromo_recursivo(texto[1:-1])

texto = 'arara'
print(palindromo_recursivo(texto))

# iteracao
def palindromo_iterativo(texto):
    n = len(texto)
    for i in range(n//2):
        if texto[i] != texto[-i-1]:
            return 'a string não é um palíndromo.'
    return 'a string é um palíndromo.'

print(palindromo_iterativo(texto))
```

```
a string é um palíndromo.
a string é um palíndromo.
```